

1、 Introduction

With the rapid development of intelligent transportation systems, accurately recognizing the status of traffic lights is of great significance for autonomous driving and traffic monitoring. This project aims to develop a machine learning-based traffic light image **classification** system that can automatically and accurately determine the color of traffic lights (red, green, yellow, off).

input: Traffic Light Images. **output:** One of the four types: Red, Yellow, Green, Off.

2、 Data Collection

Objective: Crawl images of traffic lights on the Internet as raw data for further tasks. Delete unsuitable images.

I collected data through two methods: one is by using my own written web crawler code to collect images of traffic lights on websites, and the other is by utilizing existing open-source datasets. Here is my detailed collection process:

3、 Feature Extraction

3.1.Data Annotation

In this part, I mainly use deep learning methods for feature extraction. I first manually annotated 1000 traffic light images using labelling

After the annotation was completed, I used YOLOv11 for training and automatic annotation. I then manually inspected the annotation results and, upon finding them to be accurate, continued to use YOLOv11 for annotating the original dataset. Finally, I obtained a TensorFlow dataset.

3.2 Data Augmentation

```
def load_data(img_size, batch_size):
    train_datagen = ImageDataGenerator(
        rescale=1. / 255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest'
    )
    test_datagen = ImageDataGenerator(rescale=1. / 255)
```

1. **Normalization:** By using `rescale=1./255`, the image pixel values are scaled from the range [0, 255] to [0, 1], which helps with the stability and efficiency of model training.
2. **Random Rotation:** `rotation_range=40` allows the image to be randomly rotated by up to ± 40 degrees during training, increasing the model's robustness to changes in image orientation.
3. **Horizontal and Vertical Shifts:** `width_shift_range=0.2` and `height_shift_range=0.2` allow the image to be randomly shifted horizontally and vertically by up to 20% of the pixels, helping the model learn to recover information from partial occlusions.
4. **Shear Transformation:** `shear_range=0.2` provides the functionality of shear transformation on the image, emulating perspective changes from certain angles.
5. **Zooming:** `zoom_range=0.2` allows the random zooming of the image, training the model to recognize objects of varying sizes.
6. **Horizontal Flip:** `horizontal_flip=True` randomly flips the image horizontally, which is very useful for classification tasks that do not have a directional preference (such as traffic light status classification).
7. **Fill Mode:** `fill_mode='nearest'` - After the above enhancements (such as rotation or shifting), the image may have some blank areas. The `nearest` mode fills these blanks with the nearest pixel.

Data Augmentation Examples



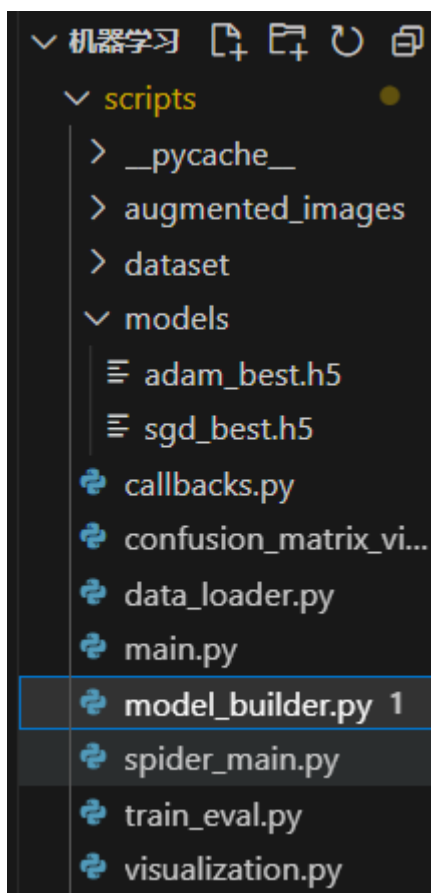
3.3.Dataset Division

We have now obtained a total of 15,839 images and divided them into training, validation, and test sets in a ratio of 75:9:16 Below is the specific data classification table.I chose this strange ratio because the number of red and green light images collected is much larger than the number of yellow and off images, so I allocated it based on a smaller number of data. The most reasonable ratio is this

	green	red	yellow	off	sum
test	1065	1438	37	102	2642
train	4789	6467	165	456	11877
val	532	719	18	51	1320
					15839

4、 Model Training

4.1 training parameters



here is a detailed description of the **training parameters**:

1. Learning Rate:

- The Adam optimizer uses a **learning rate of 0.0001**.
- The SGD optimizer uses a **learning rate of 0.01**.

2. img_size:

- The size of the image is set to **224×224** pixels. This typically means that the model expects the input images to be square with a side length of 224 pixels.

3. Batch Size:

- Set in the `data_loader.py` file, each batch contains **96** samples.

4. Number of Epochs:

- Set in the `main.py` file, the model is trained for **10** iterations over the entire dataset.

5. save_dir:

- The augmented images will be saved to a directory named "augmented_images".

6. Optimizer:

- Two optimizers are used: **Adam and SGD**.
- The Adam optimizer is known for its adaptive learning rate and momentum characteristics, suitable for most deep learning tasks.
- The SGD optimizer is a fundamental optimization algorithm that, with the introduction of a **momentum** parameter, can accelerate convergence and reduce oscillation. In this code, the momentum is **0.9**.

7. Loss Function:

- In the `train_eval.py` file, during model compilation, `sparse_categorical_crossentropy` is used as the **loss function**, suitable for multi-class classification problems where class labels are integers rather than one-hot encoded.

8. Evaluation Metrics:

- Also in the `train_eval.py` file, during model compilation, **accuracy** is set as the evaluation metric to measure the correctness of the model's predictions.

```
# 训练两个模型
model_adam, history_adam = train_model(
    train_generator, validation_generator,
    tf.keras.optimizers.Adam(learning_rate=0.0001), "adam",
    [save_augmented_cb_adam], batch_size, epochs
)
model_sgd, history_sgd = train_model(
    train_generator, validation_generator,
    tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9), "sgd",
    [save_augmented_cb_sgd], batch_size, epochs
)
```

```
# 参数设置
img_size = (224, 224)
batch_size = 96
epochs = 10
save_dir = "augmented_images"
```

4.2 Model Architecture

I utilized a Convolutional Neural Network (CNN) as the model for training, and below is a detailed description of the model architecture:

1. Input Layer:

- Input shape of (224 $\times$ 224 $\times$ 3), representing the height, width, and color channels (RGB) of the images.

2. First Layer: Convolutional Layer (Conv2D):

- Number of filters: 32
- Kernel size: 3x3
- Activation function: ReLU (Rectified Linear Unit)
- This layer is used for extracting basic features from the images.

3. Second Layer: Max Pooling Layer (MaxPooling2D):

- Pooling window size: 2x2
- This layer reduces the spatial dimensions of the feature maps while retaining the most significant features.

4. Third Layer: Convolutional Layer (Conv2D):

- Number of filters: 64
- Kernel size: 3x3
- Activation function: ReLU
- Further extracts more complex features.

5. Fourth Layer: Max Pooling Layer (MaxPooling2D):

- Pooling window size: 2x2
- Reduces the spatial dimensions of the feature maps once again.

6. Fifth Layer: Convolutional Layer (Conv2D):

- Number of filters: 128

- Kernel size: 3x3
- Activation function: ReLU
- Extracts deeper-level features.

7. Sixth Layer: Max Pooling Layer (MaxPooling2D):

- Pooling window size: 2x2
- Further reduces the spatial dimensions of the feature maps.

8. Seventh Layer: Flatten Layer:

- Converts the multi-dimensional feature maps into a one-dimensional array to be fed into the fully connected layer.

9. Eighth Layer: Fully Connected Layer (Dense):

- Number of neurons: 512
- Activation function: ReLU
- Maps the extracted features to the output classes.

10. Ninth Layer: Dropout Layer:

- Dropout rate: 0.5
- Prevents overfitting by randomly setting a fraction of input units to 0 at each update during training time.

11. Tenth Layer: Output Layer (Dense):

- Number of neurons: 4 (corresponding to four classes: green, red, yellow, off)
- Activation function: Softmax
- Used for multi-class classification, outputting the probability of each class.

```
def build_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Conv2D(128, (3, 3), activation='relu'),
        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(512, activation='relu'),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(4, activation='softmax')
    ])
    return model
```

To make the model more intuitive, I used code to draw an architectural diagram that shows the flow of data through the network, as well as the input and output shapes of each layer:

conv2d (Conv2D)

Input shape: (None, 224, 224, 3)

Output shape: (None, 222, 222, 32)



max_pooling2d (MaxPooling2D)

Input shape: (None, 222, 222, 32)

Output shape: (None, 111, 111, 32)



conv2d_1 (Conv2D)

Input shape: (None, 111, 111, 32)

Output shape: (None, 109, 109, 64)



max_pooling2d_1 (MaxPooling2D)

Input shape: (None, 109, 109, 64)

Output shape: (None, 54, 54, 64)



conv2d_2 (Conv2D)

Input shape: (None, 54, 54, 64)

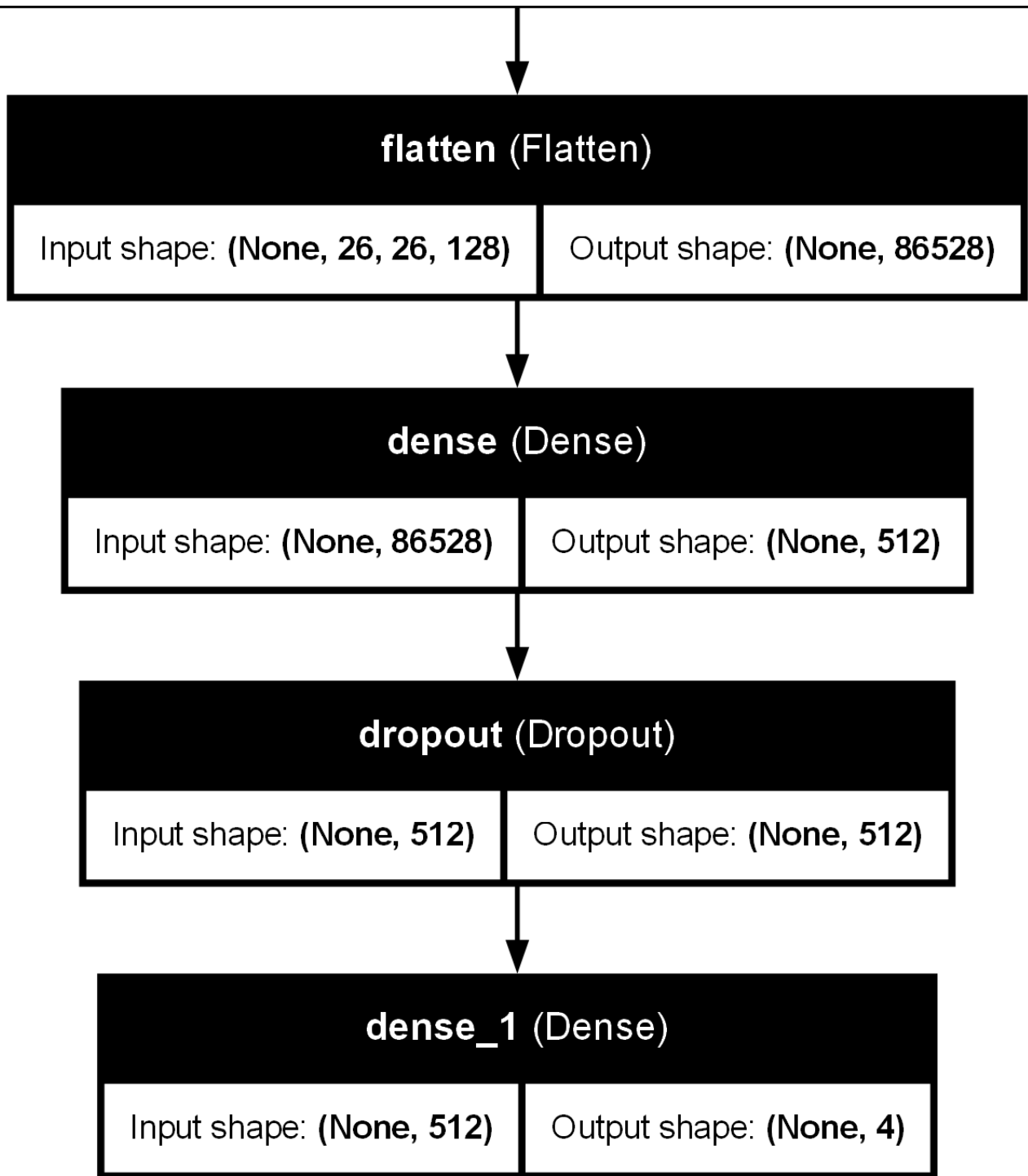
Output shape: (None, 52, 52, 128)



max_pooling2d_2 (MaxPooling2D)

Input shape: (None, 52, 52, 128)

Output shape: (None, 26, 26, 128)



Adam Training log

```

Epoch 1/10
329/329 [=====] - 211s 638ms/step - loss: 0.1793 - accuracy: 0.9380 - val_loss: 0.0499 - val_accuracy: 0.9893
Epoch 2/10
329/329 [=====] - 235s 714ms/step - loss: 0.0744 - accuracy: 0.9784 - val_loss: 0.0445 - val_accuracy: 0.9916
Epoch 3/10
329/329 [=====] - 235s 712ms/step - loss: 0.0633 - accuracy: 0.9805 - val_loss: 0.0423 - val_accuracy: 0.9886
Epoch 4/10
329/329 [=====] - 234s 711ms/step - loss: 0.0567 - accuracy: 0.9821 - val_loss: 0.0351 - val_accuracy: 0.9916
Epoch 5/10
329/329 [=====] - 232s 706ms/step - loss: 0.0491 - accuracy: 0.9851 - val_loss: 0.0383 - val_accuracy: 0.9863
Epoch 6/10
329/329 [=====] - 237s 719ms/step - loss: 0.0477 - accuracy: 0.9858 - val_loss: 0.0299 - val_accuracy: 0.9909
Epoch 7/10
329/329 [=====] - 229s 694ms/step - loss: 0.0413 - accuracy: 0.9871 - val_loss: 0.0400 - val_accuracy: 0.9924
Epoch 8/10
329/329 [=====] - 222s 675ms/step - loss: 0.0408 - accuracy: 0.9871 - val_loss: 0.0284 - val_accuracy: 0.9916
Epoch 9/10
329/329 [=====] - 229s 697ms/step - loss: 0.0422 - accuracy: 0.9876 - val_loss: 0.0283 - val_accuracy: 0.9924
Epoch 10/10
329/329 [=====] - 230s 698ms/step - loss: 0.0375 - accuracy: 0.9879 - val_loss: 0.0238 - val_accuracy: 0.9916

```

SGD Training log

```

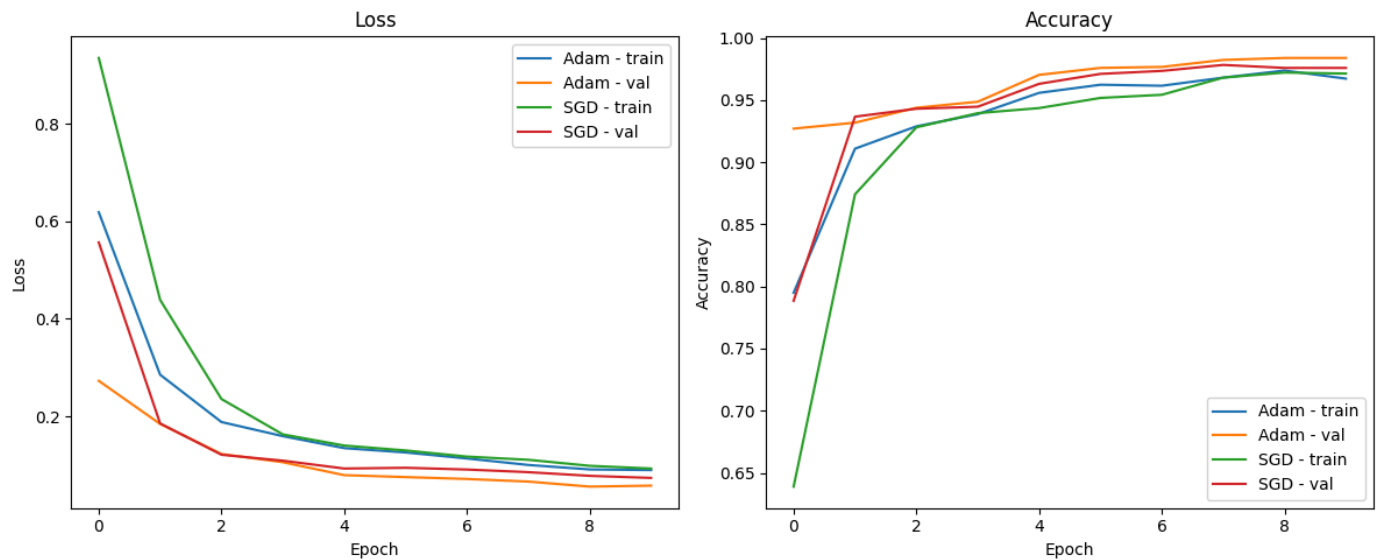
Epoch 1/10
329/329 [=====] - 190s 575ms/step - loss: 0.1580 - accuracy: 0.9459 - val_loss: 0.0797 - val_accuracy: 0.9764
Epoch 2/10
329/329 [=====] - 192s 582ms/step - loss: 0.0928 - accuracy: 0.9722 - val_loss: 0.0410 - val_accuracy: 0.9878
Epoch 3/10
329/329 [=====] - 197s 598ms/step - loss: 0.0830 - accuracy: 0.9736 - val_loss: 0.0738 - val_accuracy: 0.9855
Epoch 4/10
329/329 [=====] - 194s 588ms/step - loss: 0.0752 - accuracy: 0.9774 - val_loss: 0.0865 - val_accuracy: 0.9748
Epoch 5/10
329/329 [=====] - 193s 585ms/step - loss: 0.0687 - accuracy: 0.9808 - val_loss: 0.0308 - val_accuracy: 0.9870
Epoch 6/10
329/329 [=====] - 192s 583ms/step - loss: 0.0541 - accuracy: 0.9830 - val_loss: 0.0390 - val_accuracy: 0.9870
Epoch 7/10
329/329 [=====] - 198s 601ms/step - loss: 0.0703 - accuracy: 0.9790 - val_loss: 0.0444 - val_accuracy: 0.9916
Epoch 8/10
329/329 [=====] - 197s 600ms/step - loss: 0.0615 - accuracy: 0.9817 - val_loss: 0.0376 - val_accuracy: 0.9848
42/42 [=====] - 8s 193ms/step
42/42 [=====] - 5s 108ms/step

```

5.Result Analysis

5.1Compare of Loss and Accuracy

Comparison of Loss and Accuracy for Adam and SGD Algorithms Based on Epoch Size:



1. Convergence Speed:

- The loss graph (left) shows that SGD converges faster than Adam. In the initial epochs, SGD's loss decreases more rapidly.
- Similarly, in the accuracy graph (right), SGD shows a quicker increase in accuracy at the beginning.

2. Stability:

- Adam's loss and accuracy curves are relatively smooth, indicating a more stable training process.
- SGD's curves exhibit more fluctuation, especially in the loss graph, suggesting that SGD is more sensitive to the learning rate setting at the start of training.

3. Final Performance:

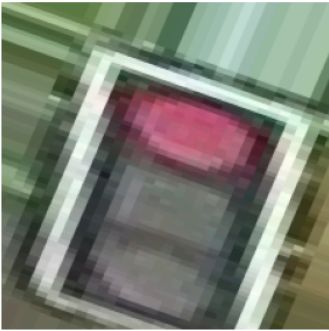
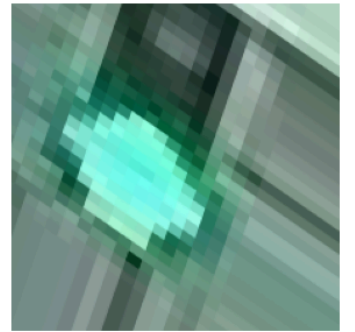
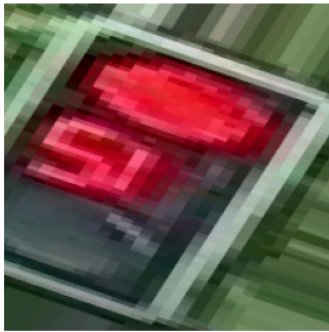
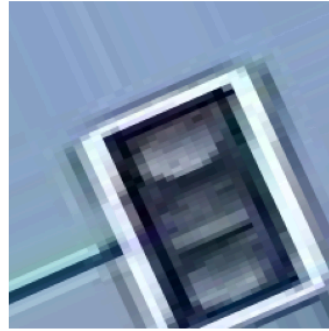
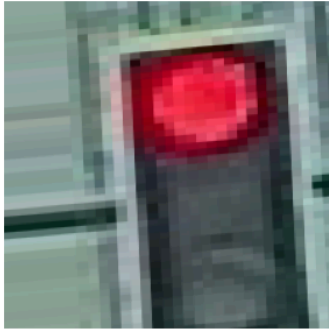
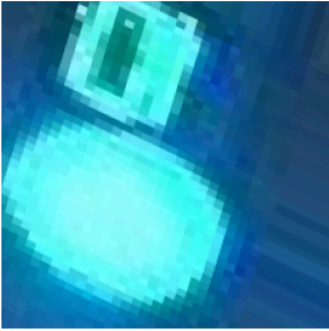
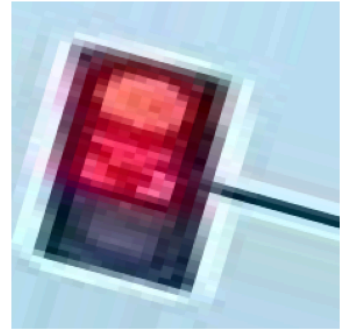
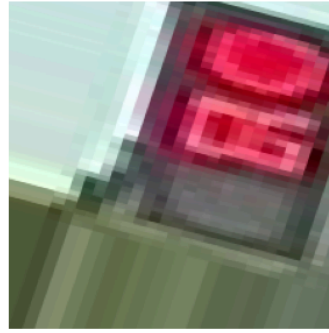
- By the end of training (around the 8th epoch), both algorithms' losses stabilize, with Adam showing a slightly lower loss, indicating a marginally better performance on the training set.
- In terms of accuracy, both algorithms achieve similar high accuracy rates on both the training and validation sets, with Adam slightly outperforming SGD on the validation set.

4. Overfitting:

- The graphs indicate that both algorithms perform closely on both the training and validation sets, with no significant overfitting observed.

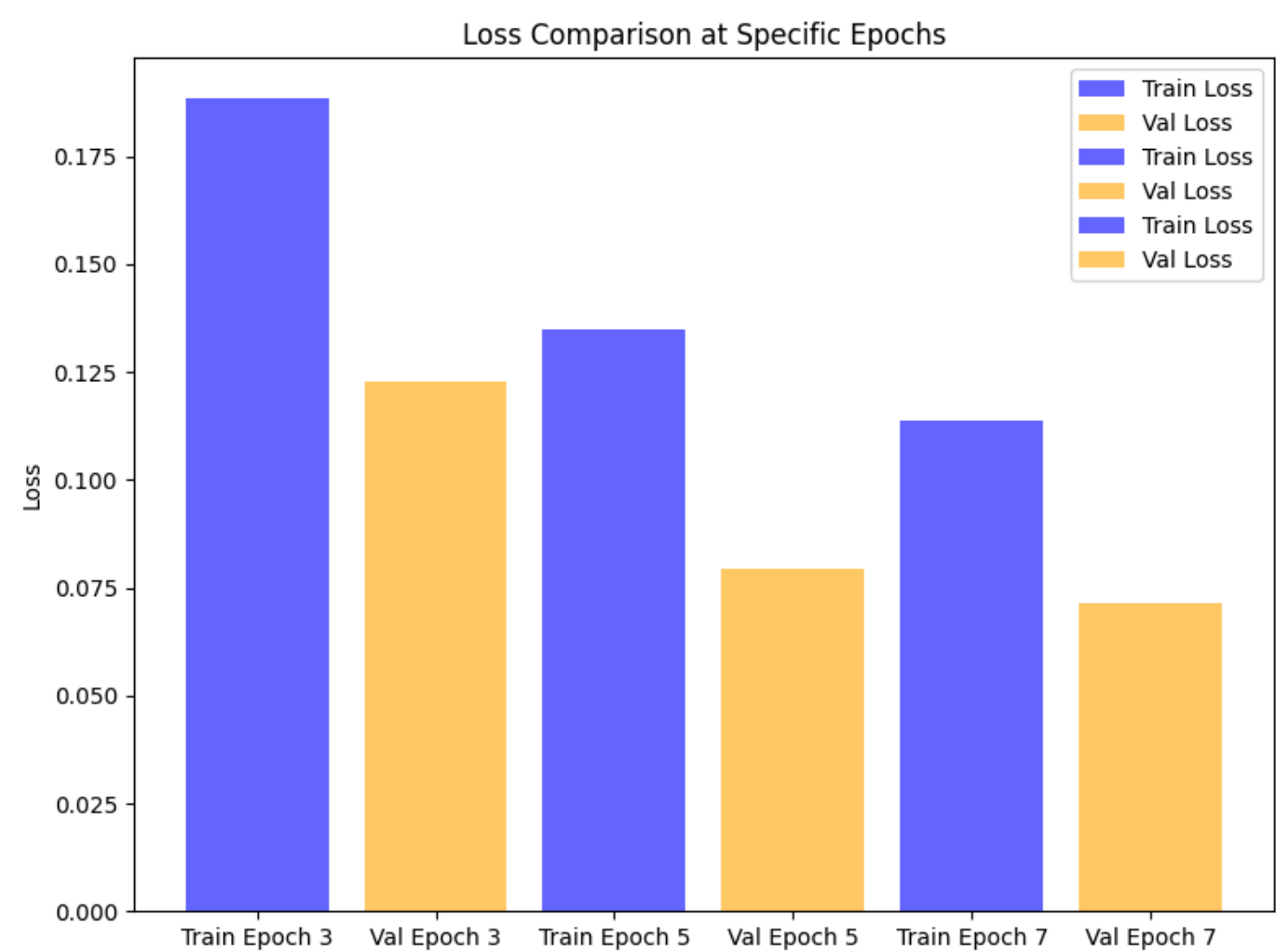
5.2 Comparison of real images

The performance of the Adam algorithm at epochs 3, 5, and 7:

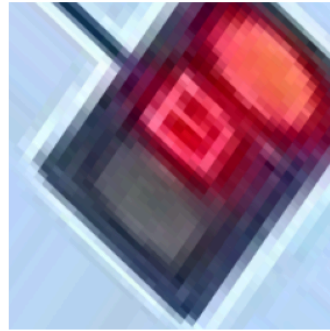
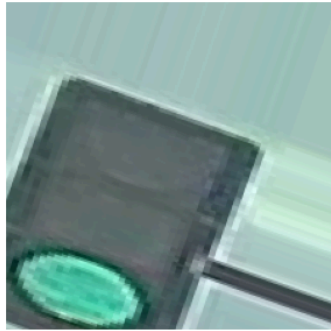
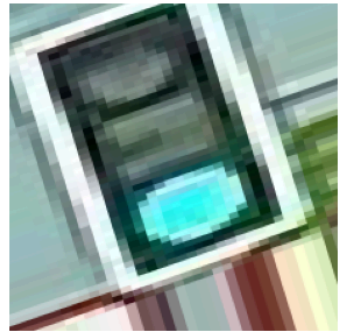
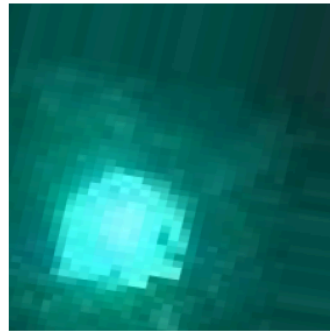
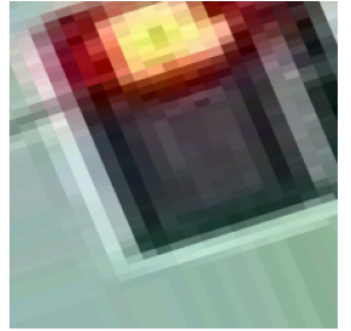
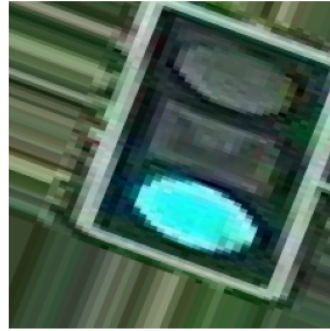
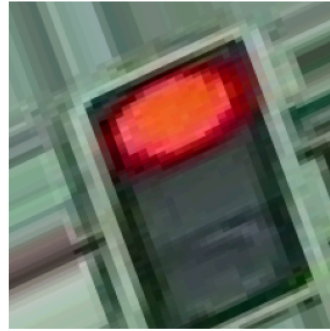
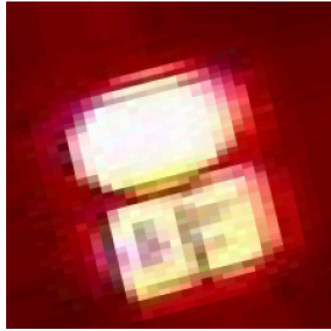


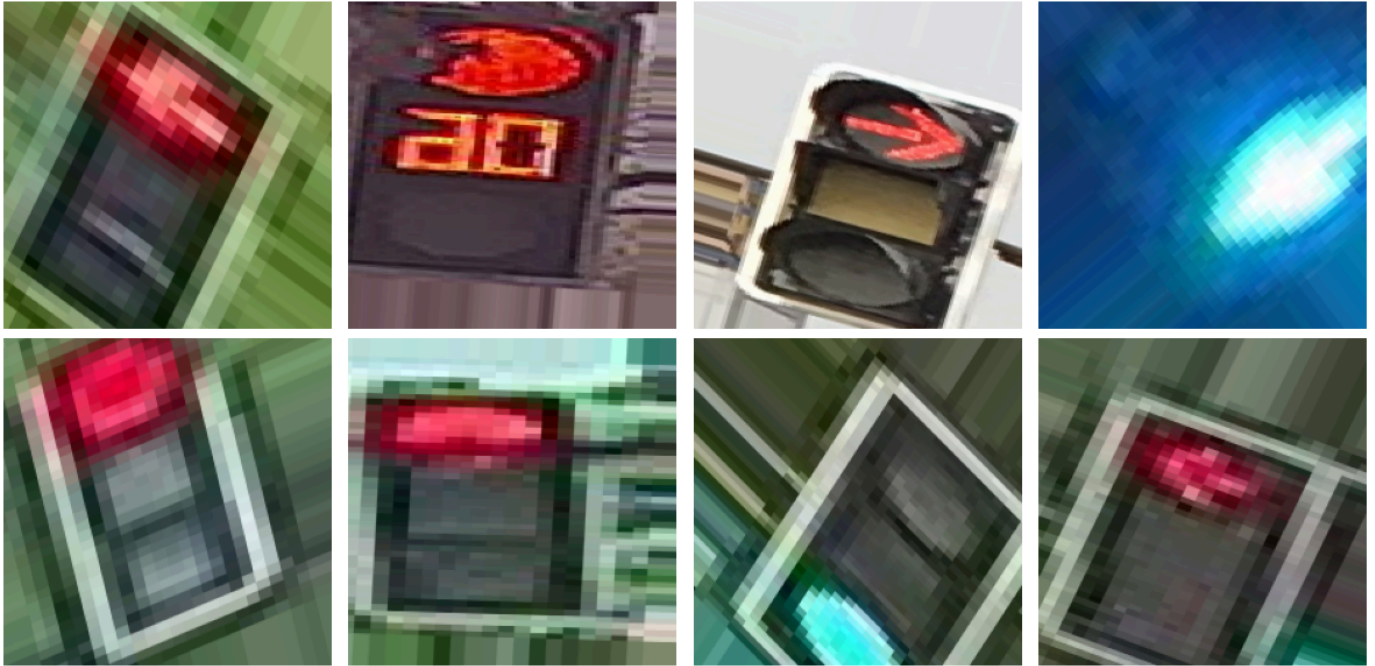


loss comparison:

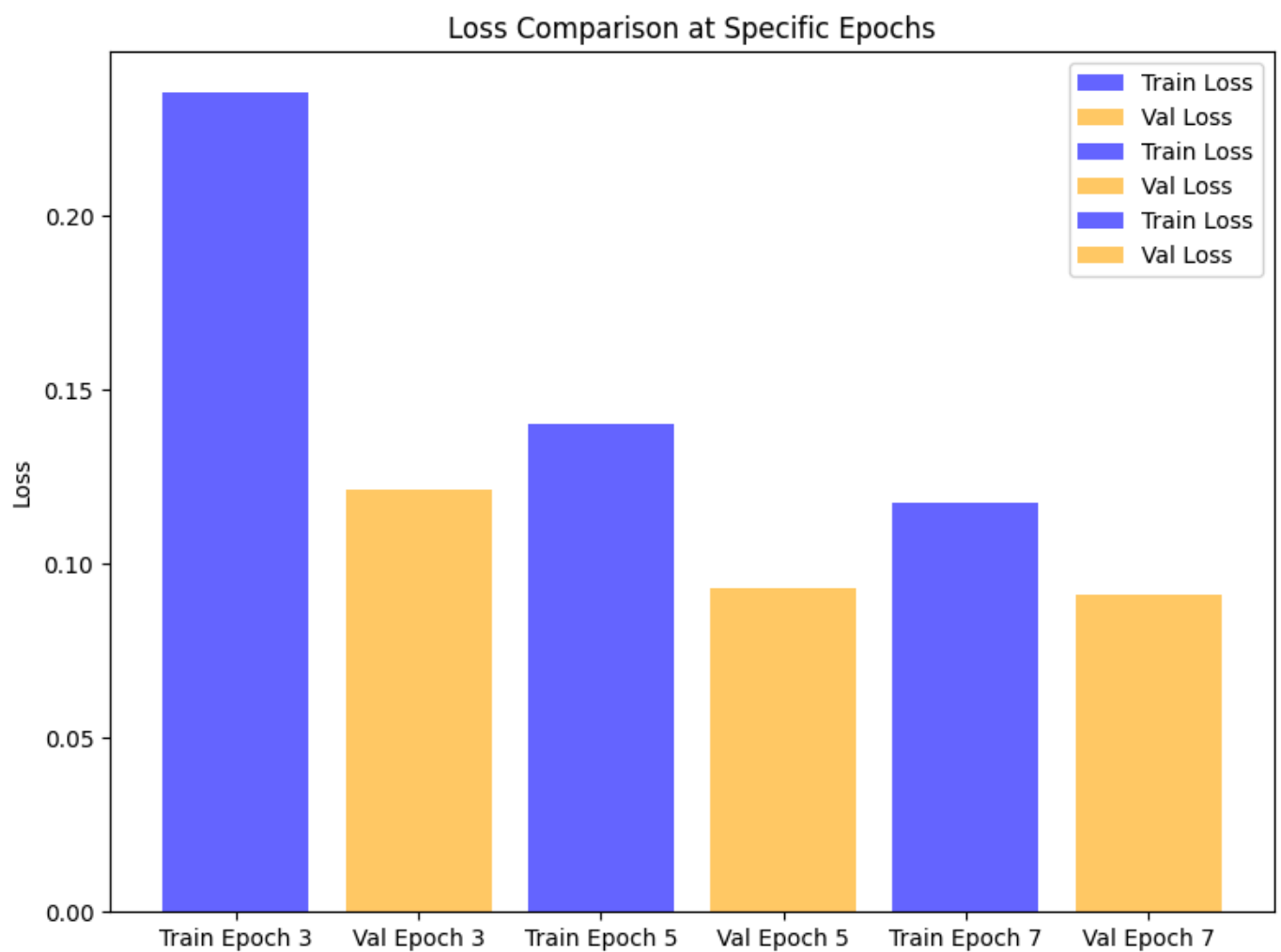


The performance of the SGD algorithm at epochs 3, 5, and 7:





loss comparison:



1. Initial Loss Values:

- At Epoch 3, for both Adam and SGD, the training loss is significantly higher than the validation loss. This might indicate that the model is not fitting well to the training data initially, or the training

data could be more complex or harder to predict than the validation data.

2. Trend of Loss Reduction:

- For the Adam optimizer, there is a significant drop in training loss from Epoch 3 to Epoch 5, followed by a plateau. The validation loss also shows a downward trend, but with a smaller reduction.
- For the SGD optimizer, the training loss decreases from Epoch 3 to Epoch 5 and then slightly increases at Epoch 7, which could suggest the onset of overfitting. The validation loss decreases between Epoch 3 and Epoch 5 and then stabilizes at Epoch 7.

3. Signs of Overfitting:

- In the SGD chart, the training loss at Epoch 7 is lower than the validation loss, which could be a sign of overfitting, where the model performs well on the training data but not as well on unseen data (validation set).
- In the Adam chart, the training and validation losses are relatively close at Epoch 7, suggesting that Adam might be generalizing better.

4. Convergence Speed:

- The Adam optimizer is known for its fast convergence, and the chart shows that by Epoch 5, the loss values are already relatively low, whereas SGD still has higher loss values at the same epoch.

5. Stability:

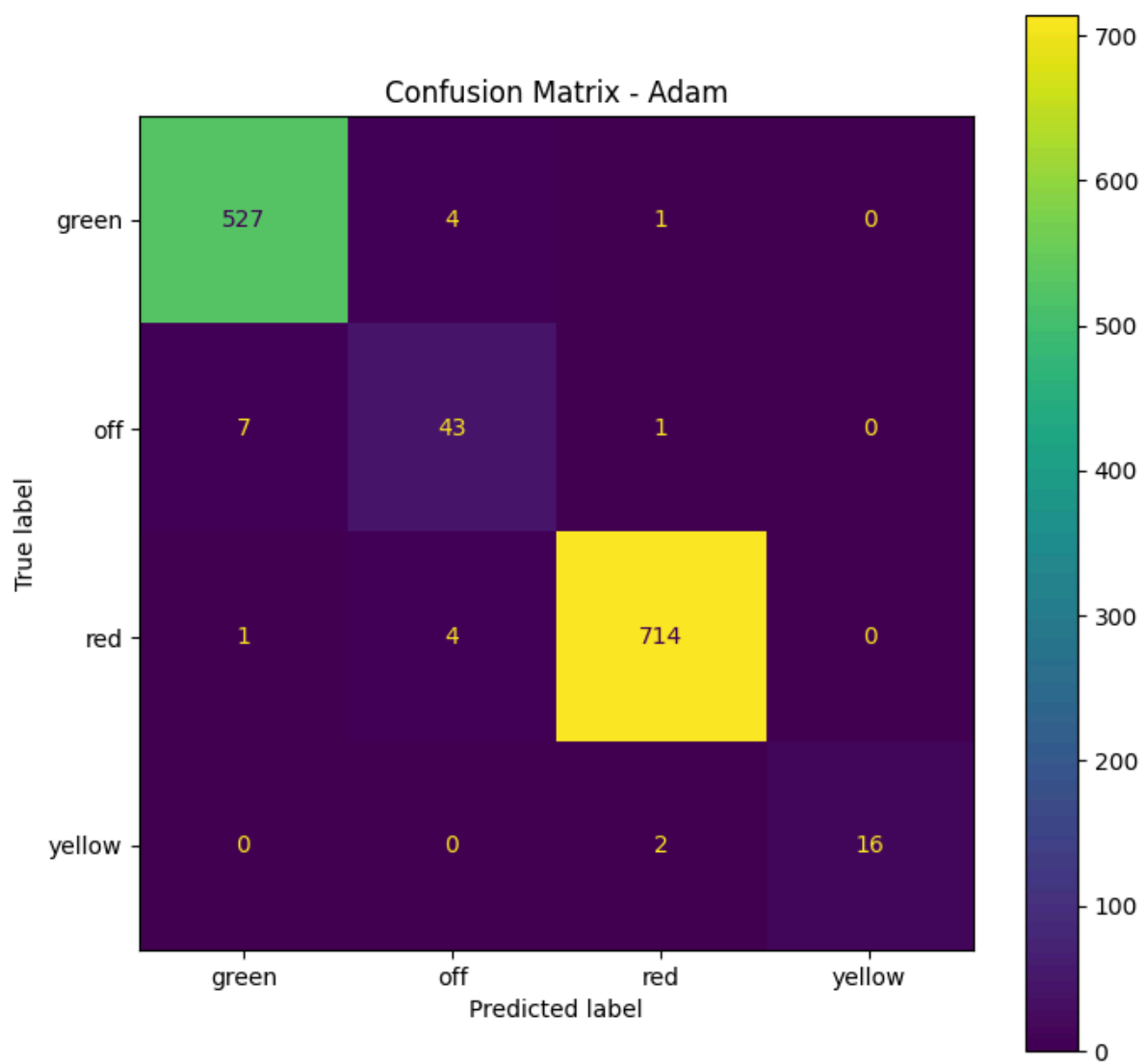
- Adam shows a more stable decrease in loss throughout the training process, while SGD's loss reduction becomes less stable after Epoch 5, which could be related to the step size of the gradient updates in SGD.

6. Final Loss Values:

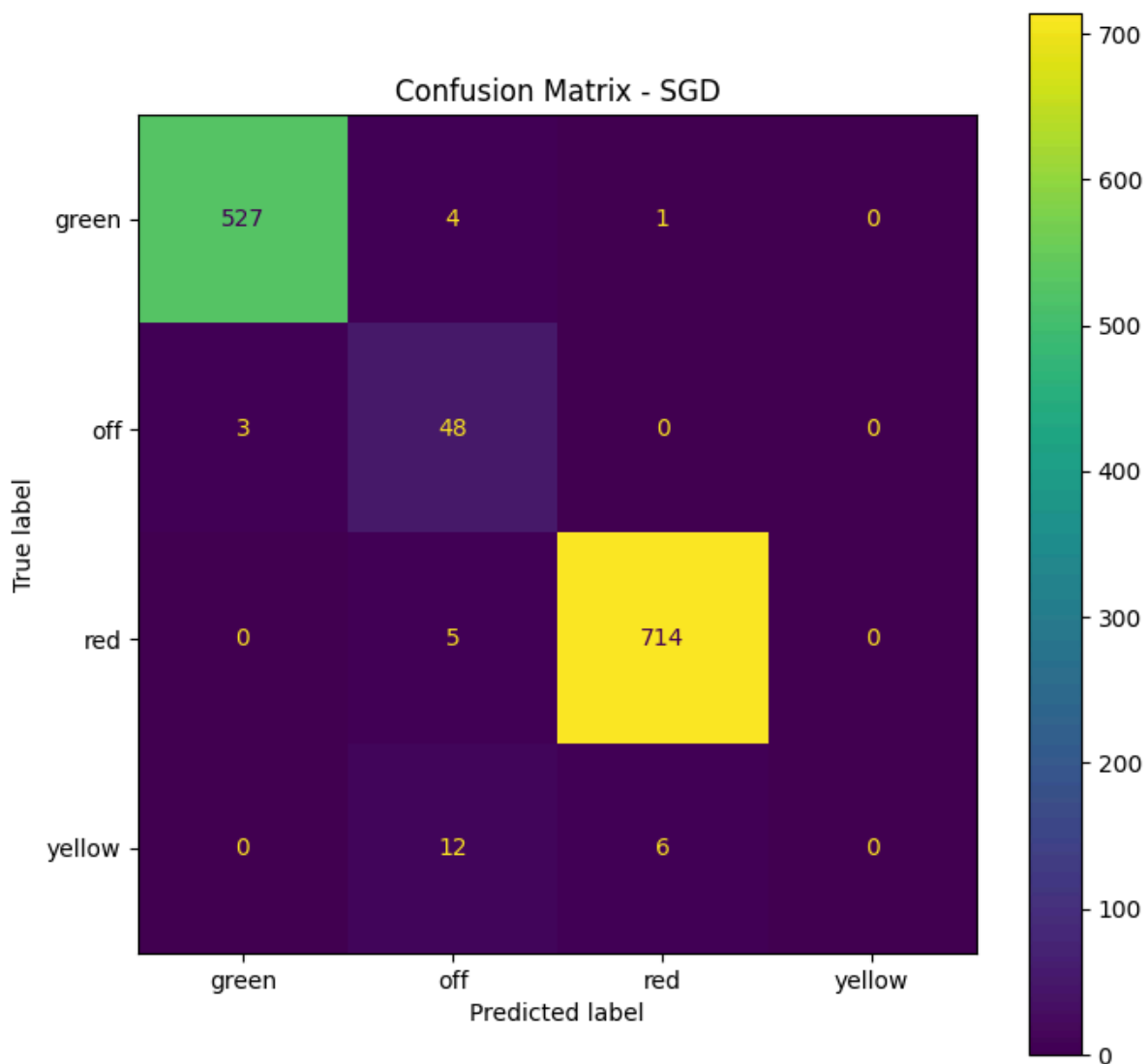
- At Epoch 7, Adam has a lower validation loss than SGD, indicating that Adam might provide better generalization performance after the same number of training epochs.

5.3 Compare of confusion matrix

Adam:



SGD:



Comparative Analysis:

- Accuracy:** Both Adam and SGD show similar accuracy for the Green, Red, and Off categories. However, SGD has a 0% accuracy for the Yellow category, indicating a significant issue with distinguishing Yellow from other categories.
- Misclassification:** Adam has fewer misclassifications for the Yellow category (2) compared to SGD, which misclassified all Yellow samples as either Off or Red, suggesting a bias in SGD's handling of Yellow samples.
- Stability:** Adam has slightly more misclassifications for the Off category (7 Green, 1 Red) than SGD (3 Green), but considering the Yellow category's performance, Adam might be more stable overall.

4. **Optimization Effectiveness:** Adam, which typically includes momentum and adaptive learning rates, may converge faster and improve the model's generalization during training. The performance on the Yellow category suggests that Adam might have performed better in this experiment.

6. Evaluation Metrics

6.1 Metrics used

In this project, I used **accuracy** and **confusion matrix** as evaluation metrics, and provided a detailed comparison and illustration **in the fifth section**.

6.2 Effort for improving the evaluation result

1. Data Augmentation (`data_loader.py`)

- The system employs data augmentation techniques such as rotation, width and height shift, shear, zoom, and horizontal flip to increase the diversity of the training data. This helps the model generalize better to new, unseen images.

2. Model Complexity (`model_builder.py`)

- A convolutional neural network (CNN) with multiple layers, including convolutional, max-pooling, flattening, dense, and dropout layers, is used. This architecture is designed to learn complex features from the images.

3. Regularization Techniques

- Dropout is included in the model to prevent overfitting by randomly setting a fraction of input units to 0 at each update during training time.

4. Callback Functions (`callbacks.py`)

- Custom callbacks are used to save augmented images at specific epochs, which can be analyzed to understand the effect of data augmentation on the model's performance.

5. Web Scraping for Data Collection (`spider_main.py`)

- Additional images are collected using web scraping to increase the dataset size and diversity, which can help improve the model's ability to generalize.

6.3 Discussion on unpredictability or bias

Unpredictability

1. **Data Diversity:** In `data_loader.py`, augmentation techniques such as rotation, scaling, and shearing are used to increase data diversity. This helps reduce the model's reliance on specific image features, thereby improving its generalization capabilities.
2. **Overfitting Risk:** In `train_eval.py`, overfitting is mitigated by using Early Stopping and Model Checkpoint callbacks. Overfitting can lead to a model that performs well on training data but poorly on unseen data, which is a source of unpredictability.
3. **Randomness:** Randomness in the training process, as seen in `train_eval.py`, can come from weight initialization, the randomness in data augmentation, and the stochastic nature of optimization algorithms (e.g., the random gradient selection in SGD). This randomness can lead to varying model behaviors across different training sessions.

Bias

1. **Dataset Bias:** there is an imbalance in the number of traffic light images across different classes, as suggested by the categories in `data_loader.py` ('green', 'off', 'red', 'yellow'), it could lead to a model that is more accurate for the majority classes and less so for the minority classes.
2. **Augmentation Bias:** The data augmentation strategy can introduce bias. For instance, if augmentation techniques are not uniformly applied across all classes, the model may become more sensitive to variations in certain classes.
3. **Model Architecture Bias:** The choice of model architecture in `model_builder.py` can affect bias. An overly complex model might learn noise in the training data rather than the underlying patterns.
4. **Training Process Bias:** The choice of hyperparameters during training, such as learning rate, batch size, and number of epochs, as seen in `train_eval.py`, can influence bias. Poor hyperparameter choices can prevent the model from learning the true distribution of the data.
5. **Optimizer Bias:** Different optimizers, as compared in `main.py` (Adam and SGD), can lead to bias in the model's learning process. For example, SGD might be more prone to getting stuck in local minima compared to Adam.